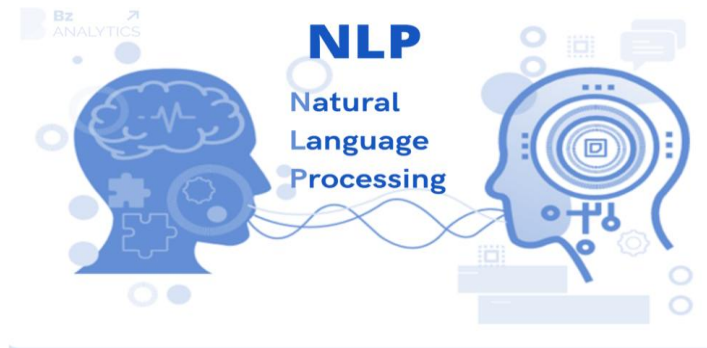


Digilians



NLP

Session 1

Session Agenda

- ☐ What is NLP
- ☐ NLP Pipeline
- ☐ Regular Expression

What is NLP ?

What is Natural Language Processing?

Natural language processing (NLP) is a subfield of computer science and **artificial intelligence (AI)** that uses machine learning to enable computers to understand and communicate with human language.

What is Natural Language Processing?

Applications In NLP :

Text Classification 

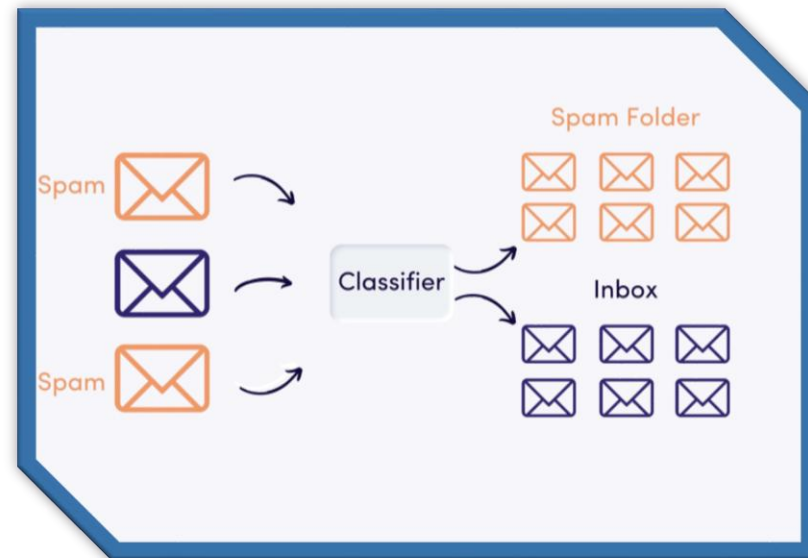
Sentiment Analysis 

Text Similarity 

Text Summarization 

Machine Translation 

Chatbots 



What is Natural Language Processing?

There are some terms we need to agree on:

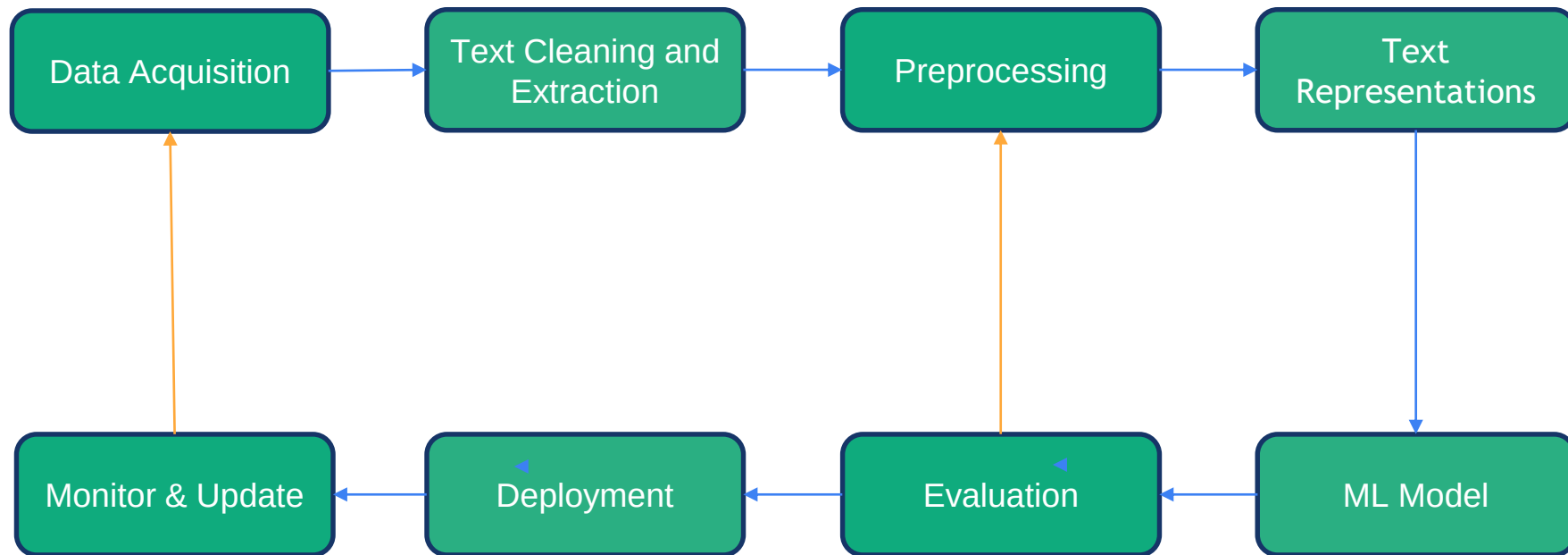
Corpus → Paragraph

Document → Sentences

Vocabulary → Unique Words

NLP Pipeline

NLP Pipeline



NLP Pipeline

Data Acquisition

- CSV & Excel
- Database
- JSON
- Text Files
- PDF

NLP Pipeline

Data Acquisition

Text Cleaning and
Extraction

```
{  
  "title": "camtasia closing and crashing during batch production",  
  "description": "So here's another post complaining about something I've  
    read a couple of times before:\n\n    During video production  
    (though mostly batch production as that's what I usually use),  
    Camtasia loves to cras. The crash is basically..",  
  "creator": "Vladimir Putin",  
  "create_ts": "3/23/2022 10:30 AM EST",  
  "severity": "High"  
}
```

```
{  
  "text": "camtasia closing and crashing during batch production. So here's  
    another post complaining about something I've read a couple of  
    times before:\n\n    During video production (though mostly batch  
    production as that's what I usually use), Camtasia loves to crash.  
    The cras is basically..",  
  "severity": "High"  
}
```

Regular Expressions

Regular expression

Text Cleaning using Regex:

We use it to extract a specific pattern from the text.

This can be implemented in vision using something called optical character recognition (OCR).

Of course, it can't be saved, so that's why we're going to this website <https://regex 101.com/>

Reg[ular]
Ex[pression]

Regular expression

Python has a built-in package called re, which can be used to work with Regular Expressions.
Import the re module:

```
import re
```

Regular expression

RegEx Functions



<i>Function</i>	Description
<i>findall</i>	Returns a list containing all matches
<i>search</i>	Returns a Match object if there is a match anywhere in the string
<i>split</i>	Returns a list where the string has been split at each match
<i>sub</i>	Replaces one or many matches with a string

Regular expression

The findall() Function

The *findall()* function returns a list containing all matches.

```
1 import re
2 txt = "The Course in Egypt is one of the best places. (Course)"
3 x = re.findall("Course", txt)
4 print(x)
```

['Course', 'Course']

If no matches are found, **an empty list is returned**

```
1 import re
2 txt = "The AI for everyone"
3 x = re.findall("Course", txt)
4 print(x)
```

[]

Regular expression

The sub() Function

The *sub()* function replaces the matches with the text of your choice

```
1 import re
2 txt = "The Course in Egypt is one of the best places. (Course)"
3 x = re.sub("o", "9", txt)
4 print(x)
```

The C9urse in Egypt is 9ne 9f the best places. (C9urse)

You can control the number of replacements by specifying the **count** parameter

```
1 import re
2 txt = "The Course in Egypt is one of the best places. (Course)"
3 x = re.sub("o", "9", txt, 1)
4 print(x)
5
```

The C9urse in Egypt is one of the best places. (Course)

Regular expression

The search() Function

The ***search()*** function searches the string for a match and returns a Match object if there is a match. If there is more than one match, only the first occurrence of the match will be returned:

```
import re

txt = "The rain in Spain"
x = re.search("\s", txt)

print("The first white-space character is located in position:", x.start())
```

The first white-space character is located in position: 3

If no matches are found, the value **None** is returned:

```
import re

txt = "The rain in Spain"
x = re.search("Portugal", txt)
print(x)
```

None

Regular expression

Match ranges in regex

Regex uses ranges to match sequential groups of characters, like uppercase English letters, without having to write out each letter individually. This makes it easier and less tedious to specify such patterns in your expressions.

RANGE	MATCHES
[A-Z]	uppercase letters
[a-z]	lowercase letters
[0-9]	Any digit

/regex?/

Regular expression

Examples

```
import re
input_string = 'Hello World 123'

uppercase_letters = re.findall(r'[A-Z]', input_string)
print("Uppercase letters:", uppercase_letters)
```

Uppercase letters: ['H', 'W']

```
import re
input_string = 'Hello World 123'

# Lowercase letters
lowercase_letters = re.findall(r'[a-z]', input_string)
print("Lowercase letters:", lowercase_letters)
```

Lowercase letters: ['e', 'l', 'l', 'o', 'o', 'r', 'l', 'd']

Regular expression

Examples

```
import re
input_string = 'Hello World 123'

# Digits
digits = re.findall(r'[0-9]', input_string)
print("Digits:", digits)
```

Digits: ['1', '2', '3']

```
1 import re
2
3 text = "There are 3 apples, 12 bananas, and 7 oranges."
4 matches = re.findall(r'[0-3]', text)
5 print(matches)
6
```

['3', '1', '2']

Regular expression

Special Sequences

RANGE	MATCHES
.	anything except newline
\w	word character
\W	non-word character
\d	digits
\D	non-digits
\s	space, tab, newlines
\S	non whitespace characters

Regular expression

Examples

```
import re

text = "Hello World_123"

pattern = re.compile(r'\w')
matches = pattern.findall(text)
print("Matches any word character :", matches)
```

Matches any word character : ['H', 'e', 'l', 'l', 'o', 'W', 'o', 'r', 'l', 'd', '_', '1', '2', '3']

```
import re

# Sample text
text = "Hello World_123!"

# Match non-word characters
pattern = re.compile(r'\W')
matches = pattern.findall(text)
print("Match non-word characters", matches)
```

Match non-word characters [' ', '!']

Regular expression

Examples

```
import re
```

```
# Sample text
```

```
text = "Hello 123 World 456!"
```

```
# Match non-digits
```

```
pattern = re.compile(r'\D')
```

```
matches = pattern.findall(text)
```

```
print(matches)
```

```
['H', 'e', 'l', 'l', 'o', ' ', ' ', ' ', 'W', 'o', 'r', 'l', 'd', ' ', ' ', '!']
```

```
import re
```

```
# Sample text
```

```
text = "Hello 123 World 456"
```

```
# Match digits
```

```
pattern = re.compile(r'\d')
```

```
matches = pattern.findall(text)
```

```
print(matches)
```

```
['1', '2', '3', '4', '5', '6']
```


Regular expression

Examples

```
import re

# Sample text
text = "Hello World Python"

# Match non-whitespace characters
pattern = re.compile(r'\S')
matches = pattern.findall(text)
print(matches)
```

['H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd', ' ', 'P', 'y', 't', 'h', 'o', 'n']

```
import re

# Sample text
text = "Hello\tWorld\nPython"

# Match whitespace characters
pattern = re.compile(r'\s')
matches = pattern.findall(text)
print(matches)
```

['\t', '\n']

```
import re
text = "Hello, world\n!"
pattern = r".,!"
matches = re.findall(pattern, text)
print(matches)
```

['H', 'e', 'l', 'l', 'o', ',', ' ', ' ', 'w', 'o', 'r', 'l', 'd', '!', '\n']

Regular expression

Metacharacters

RANGE	MATCHES
*	0 or more
?	0 or 1
+	1 or more
{n}	exactly n times
{n, }	n or more times
{n, m}	n to m times inclusive

/regex?/

Regular expression

Examples

```
import re

pattern = r"ab*"
test_string = "ababab"
result = re.findall(pattern, test_string)
print(result)  # Output: ['ab', 'ab', 'ab']
```

['ab', 'ab', 'ab']

```
import re

pattern = r"ab+"
test_string = "abb"
result = re.findall(pattern, test_string)
print(result)  # Output: ['abb']
# +: Matches one or more occurrences of the preceding element.
```

['abb']

Regular expression

Examples

```
import re

pattern = r"ab?"
test_string = "a"
result = re.findall(pattern, test_string)
print(result) # Output: ['a']
```

?: Matches zero or one occurrence of the preceding element.

['a']

```
import re

pattern = r"ab{2}"
test_string = "abb"
result = re.findall(pattern, test_string)
print(result) # Output: ['abb']
```

{n}: Matches exactly n occurrences of the preceding element.

['abb']

Regular expression

Examples

```
import re

pattern = r"ab{2,3}"
test_string = "abbb"
result = re.findall(pattern, test_string)
print(result) # Output: ['abb']
# {n, m}: Matches between n and m occurrences of the preceding element, inclusive.
```

['abb']

```
import re

pattern = r"ab{2,}"
test_string = "abbbbb"
result = re.findall(pattern, test_string)
print(result) # Output: ['abbbbb']
# {n,}: Matches n or more occurrences of the preceding element.
```

['abbbbb']

Regular expression

Try To Solve

Write a regular expression to match a phone number in the
format "(XXX) XXX-XXXX"
From "call me at (123) 456-7890"

Regular expression

Try To Solve

Write a regular expression to match a phone number in the
format "(XXX) XXX-XXXX"
From "call me at (123) 456-7890"

```
import re

phone_number_pattern = r"\(\d{3}\) \d{3}-\d{4}"
test_string = "Call me at (123) 456-7890"
matches = re.findall(phone_number_pattern, test_string)
print(matches)
```

Regular expression

Try To Solve

How can you extract all email addresses from a given text
using regular expressions in Python?

“Contact us at email@example.com or info@example.com”

Regular expression

Try To Solve

How can you extract all email addresses from a given text using regular expressions in Python?

“Contact us at email@example.com or info@example.com”

```
import re

text = 'Contact us at email@example.com or info@example.com'
pattern = r'\b\w+@\w+\.'
emails = re.findall(pattern, text)
print(emails)
```

Regular expression

Python Regular Expressions

SPECIAL CHARACTERS

- `^` | Matches the expression to its right at the start of a string. It matches every such instance before each `\n` in the string.
- `$` | Matches the expression to its left at the end of a string. It matches every such instance before each `\n` in the string.
- `.` | Matches any character except line terminators like `\n`.
- `\` | Escapes special characters or denotes character classes.
- `A|B` | Matches expression `A` or `B`. If `A` is matched first, `B` is left untried.
- `+` | Greedily matches the expression to its left 1 or more times.
- `*` | Greedily matches the expression to its left 0 or more times.
- `?` | Greedily matches the expression to its left 0 or 1 times. But if `?` is added to qualifiers (`+`, `*`, and `?` itself) it will perform matches in a non-greedy manner.
- `{m}` | Matches the expression to its left `m` times, and not less.
- `{m,n}` | Matches the expression to its left `m` to `n` times, and not less.
- `{m,n}?` | Matches the expression to its left `m` times, and ignores `n`. See `?` above.

- `\A` | Matches the expression to its right at the absolute start of a string whether in single or multi-line mode.
- `\Z` | Matches the expression to its left at the absolute end of a string whether in single or multi-line mode.

SETS

- `[ ]` | Contains a set of characters to match.
- `[amk]` | Matches either `a`, `m`, or `k`. It does not match `amk`.
- `[a-z]` | Matches any alphabet from `a` to `z`.
- `[a\^-z]` | Matches `a`, `-`, or `z`. It matches `-` because `\` escapes it.
- `[a-]` | Matches `a` or `-`, because `-` is not being used to indicate a series of characters.
- `[-a]` | As above, matches `a` or `-`.
- `[a-z0-9]` | Matches characters from `a` to `z` and also from `0` to `9`.
- `[+*]` | Special characters become literal inside a set, so this matches `(.,+,*,and.)`.
- `^ab5` | Adding `^` excludes any character in the set. Here, it matches characters that are not `a`, `b`, or `5`.

GROUPS

- `( )` | Matches the expression inside the parentheses and groups it.
- `(?)` | Inside parentheses like this, `?` acts as an extension notation. Its meaning depends on the character immediately to its right.
- `(?PAB)` | Matches the expression `AB`, and it can be accessed with the group name.
- `(?aiLmsux)` | Here, `a`, `i`, `L`, `m`, `s`, `u`, and `x` are flags:
 - `a` — Matches ASCII only
 - `i` — Ignore case
 - `L` — Locale dependent
 - `m` — Multi-line
 - `s` — Matches all
 - `u` — Matches unicode
 - `x` — Verbose

POPULAR PYTHON RE MODULE

FUNCTIONS

- `re.findall(A, B)` | Matches all instances of an expression `A` in a string `B` and returns them in a list.
- `re.search(A, B)` | Matches the first instance of an expression `A` in a string `B`, and returns it as a re match object.
- `re.split(A, B)` | Split a string `B` into a list using the delimiter `A`.
- `re.sub(A, B, C)` | Replace `A` with `B` in the string `C`.

- `(?:A)` | Matches the expression as represented by `A`, but unlike `(?PAB)`, it cannot be retrieved afterwards.
- `(?#...)` | A comment. Contents are for us to read, not for matching.
- `A(?:=B)` | Lookahead assertion. This matches the expression `A` only if it is followed by `B`.
- `A(?:!B)` | Negative lookahead assertion. This matches the expression `A` only if it is not followed by `B`.
- `(?<=B)A` | Positive lookbehind assertion. This matches the expression `A` only if `B` is immediately to its left. This can only match fixed length expressions.
- `(?<!B)A` | Negative lookbehind assertion. This matches the expression `A` only if `B` is not immediately to its left. This can only match fixed length expressions.
- `(?P=name)` | Matches the expression matched by an earlier group named "name".
- `(...)\1` | The number `1` corresponds to the first group to be matched. If we want to match more instances of the same expression, simply use its number instead of writing out the whole expression again. We can use from `1` up to `99` such groups and their corresponding numbers.

Regular expression

Cheat sheet on Python RegEx:

<https://www.dataquest.io/wp-content/uploads/2019/03/python-regular-expressions-cheat-sheet.pdf>

THANK
you